Práctica 2

Representación de problemas y búsqueda no informada

Grupo 2
Hernan Velarde Quiros
Trajano Arthur Iacob

14 de Octubre de 2026

Ejercicio 1. ¿Qué algoritmo tiene mayor coste en el tiempo? Incluye una tabla en la que se muestre el nombre del algoritmo, tamaño del tablero y coste en el tiempo. Razona por qué tiene mayor coste en el tiempo.

Solución. En el problema de DomReina, el algoritmo con mayor coste en el tiempo es BCU (Búsqueda de Coste Uniforme) ya que se implementa utilizando una cola de prioridad. Esto tiene un coste de $\mathcal{O}(|frontier|)$ por cada hijo.

Tablero	Algoritmo	Tiempo
3×3	BPI[1]	0,042 ms
	BPP[2]	0,061 ms
	BPL[3]	0,068 ms
	BPA[4]	0,159 ms
	BCU[5]	0,801 ms
4×4	BPL	0,144 ms
	BPI	0,157 ms
	BPP	0,159 ms
	BPA	1,311 ms
	BCU	34,164 ms
5×5	BPL	0,000 182 s
	BPP	0,000 424 s
	BPI	0,002 338 s
	BPA	0,052 176 s
	BCU	40,760 226 s
6×6	BPP	0,000 400 s
	BPL	0,000 478 s
	BPI	0,004 888 s
	BPA	0,310 523 s
	BCU	720,951 116 s

Cuadro 1: Tiempos en Queen Domination y ordenados por rapidez

Algoritmo	Tiempo (ms)
Avara/Greedy	0,138
A*	0,155
BPA en grafo	0,222
BPI	0,556
BPA en árbol	1,122
BPL	1,169
BCU	1,271
BPP en árbol	No termina

Cuadro 2: Tiempos en el Eight puzzle ordenado por rapidez

Solución. Sin embargo, para el problema de Eightpuzzle, cada acción tiene una acción inversa inmediata, y como BPP en árbol no guarda registro de estados visitados, queda atrapado en una rama infinita de oscilaciones periódicas, haciendo que el tiempo de ejecución sea infinito.

[1] BPI: Búsqueda en Profundidad Iterativa.
[2] BPP: Búsqueda en Profundidad (Depth-First).
[3] BPL: Búsqueda en Profundidad Limitada.
[4] BPA: Búsqueda en Anchura (Breadth-First).
[5] BCU: Búsqueda de Coste Uniforme.

Ejercicio 2. ¿Encuentran todos los algoritmos solución al problema? Incluye una tabla en la que se muestre el nombre del algoritmo, tamaño del tablero y solución al problema (número mínimo de reinas). Razona por qué hay algoritmos que no encuentran solución al problema.

Solución. Por un lado tenemos búsquedas completas, si existe solución siempre se encuentra, como la busqueda por anchura y la busqueda de coste uniforme. Esta ultima por ejemplo, explota respecto al tamaño del tablero. Vemos que en $N = 3$ tarda 8 ms pero en $N = 6$ tarda 12 minutos, por lo que no encontrará una solución en un tiempo aceptable con N mayores, pero al ser completos garantizan solución en tiempo finito.

Por otro lado tenemos búsquedas no completas, que no garantizan solución, pero debido que tenemos definido un espacio de estados finito entonces son completos, es decir, si por cada nivel de profundidad se añade una pieza al tablero, en algún punto se hallará una solución o se rellenará el tablero al completo y por ende, será una solución. Entonces, estas búsquedas no completas (búsquedas de profundidad) garantizan una solución.

Tablero	Algoritmo	Reinas
3×3	BPI	1
	BPA	1
	BCU	1
	BPP	2
	BPL	2
4×4	BPI	2
	BPA	2
	BCU	2
	BPL	3
	BPP	3
5×5	BPI	3
	BPA	3
	BCU	3
	BPL	4
	BPP	4
6×6	BPI	3
	BPA	3
	BCU	3
	BPP	5
	BPL	5

Cuadro 3: Número de reinas colocadas por tamaño de tablero

Ejercicio 3. ¿Encuentran todos los algoritmos la misma solución? Razona por qué hay algoritmos con soluciones distintas.

Solución. No, no todos los algoritmos encuentran la misma solución. Tanto en el proyecto DomReina como en Eightpuzzle, los algoritmos devuelven soluciones distintas. Esta discrepancia se manifiesta en dos aspectos fundamentales: Las diferencias en optimalidad, ya que unos algoritmos encuentran la solución óptima y otros encuentran soluciones subóptimas. Y las diferencias en la configuración exacta incluso entre algoritmos con la misma profundidad, el orden en que se visita el árbol conduce a estados objetivos completamente distintos. Se dan algoritmos con soluciones distintas por las principales razones: Garantía de optimalidad según la estrategia de expansión, espacio con múltiples estados objetivo válidos y simetrías y una presencia de ciclos en el espacio de estados.

Ejercicio 4. ¿Todos los algoritmos encuentran una solución óptima al problema? Realiza una tabla en la que se muestre el nombre del algoritmo, tamaño del tablero, número mínimo de reinas colocadas y óptima (con valor SI o NO). Razona por qué hay algoritmos que no encuentran solución óptima.

Solución. Hay algoritmos que no encuentran solución óptima porque hay algunos como BPP y BPL que no exploran por niveles, lo que hace que se saturen rápidamente. Por ejemplo, en un tablero 4x4 es muy fácil cubrir todas las casillas colocando 3 reinas en posiciones consecutivas. Pero en el momento que estos algoritmos colocan la tercera reina, la función **goal_test** se evalúa a True. Como el algoritmo solo busca cualquier camino que satisfaga el objetivo, se detiene de inmediato y devuelve esa solución, sin llegar a explorar las ramas del nivel 2 donde se encontraban las soluciones óptimas con solo 2 reinas.

Tablero	Mínimo de reinas	Algoritmo	Óptima
3×3	1	BPI	SI
		BPA	SI
		BCU	SI
	2	BPP	NO
		BPL	NO
4×4	2	BPI	SI
		BPA	SI
		BCU	SI
	3	BPL	NO
		BPP	NO
5×5	3	BPI	SI
		BPA	SI
		BCU	SI
	4	BPL	NO
		BPP	NO
6×6	3	BPI	SI
		BPA	SI
		BCU	SI
	5	BPL	NO
		BPP	NO

Cuadro 4: Resultados del problema Queen Domination agrupados por tamaño de tablero y número de reinas colocadas

```
PS C:\Users\arthur\Documents\GitHub\proyectosFIA> & c:\python314\python.exe c:/Users/arthur/Documents/Git
Hub/proyectosFIA/eightpuzzle/src/eightpuzzle/main.py
Inicio del problema: (1, 5, 2, 4, 3, 0, 7, 8, 6)
Acciones posibles: ['UP', 'DOWN', 'LEFT']
Estados siguientes generados despues de aplicar las acciones:
Estado 0, accion: UP: (1, 5, 0, 4, 3, 2, 7, 8, 6)
Estado 1, accion: DOWN: (1, 5, 2, 4, 3, 6, 7, 8, 0)
Estado 2, accion: LEFT: (1, 5, 2, 4, 0, 3, 7, 8, 6)
Comprobacion del objetivo:
El estado (1, 5, 0, 4, 3, 2, 7, 8, 6)  es objetivo: False
El estado (1, 5, 2, 4, 3, 6, 7, 8, 0)  es objetivo: False
El estado (1, 5, 2, 4, 0, 3, 7, 8, 6)  es objetivo: False
PS C:\Users\arthur\Documents\GitHub\proyectosFIA>
```

Figura 1: Salida del programa Eightpuzzle

```
PS C:\Users\arthur\Documents\GitHub\proyectosFIA> & c:\python314\python.exe c:/Users/arthur/Documents/Git
Hub/proyectosFIA/reinas/src/reinas/main.py
Algoritmo: BPA
Tablero: 4x4
Coste de tiempo: 0.0028722286224365234
Numero minimo de reinas: 2
Reinas   | Fila |  Col |
Reina 1  |    0 |    0 |
Reina 2  |    2 |    2 |

Algoritmo: BPP
Tablero: 4x4
Coste de tiempo: 0.0001842975616455078
Numero minimo de reinas: 3
Reinas   | Fila |  Col |
Reina 1  |    3 |    3 |
Reina 2  |    3 |    2 |
Reina 3  |    3 |    1 |

Algoritmo: BPI
Tablero: 4x4
Coste de tiempo: 0.0002276897430419922
Numero minimo de reinas: 2
Reinas   | Fila |  Col |
Reina 1  |    0 |    0 |
Reina 2  |    2 |    2 |

Algoritmo: BCU
Tablero: 4x4
Coste de tiempo: 0.02920079231262207
Numero minimo de reinas: 2
Reinas   | Fila |  Col |
Reina 1  |    0 |    0 |
Reina 2  |    2 |    2 |

Algoritmo: BPL
Tablero: 4x4
Coste de tiempo: 0.0002624988555908203
Numero minimo de reinas: 3
Reinas   | Fila |  Col |
Reina 1  |    0 |    0 |
Reina 2  |    0 |    1 |
Reina 3  |    0 |    2 |
```

Figura 2: Salida del problema DomReina editada para includir todo en la misma imagen